

JAVA SWING LAYOUT-MANAGER ENTSCHEIDUNGSFINDER (V2)

Erweiterte Version mit Fokus auf verschachtelte Layouts (Nested Layouts)

DAS PRINZIP DER VERSCHACHTELUNG: Komplexe Oberflächen werden in Java Swing niemals mit nur einem einzigen Layout-Manager gebaut. Man baut kleine, in sich geschlossene JPanel-Objekte mit einem einfachen Layout (z. B. GridLayout für die Eingabe oder FlowLayout für Knöpfe) und setzt diese Teil-Panels dann als Komponenten in ein übergeordnetes Panel (z. B. den JFrame mit BorderLayout).

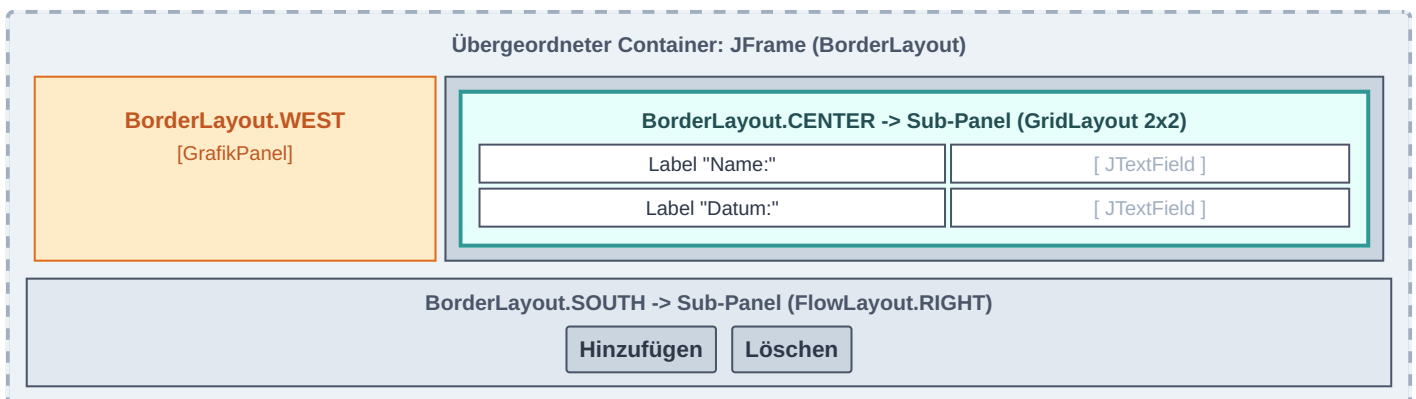
1. Schnellübersicht der Basis-Layouts

Layout	Typischer Klausureinsatz	Wichtiges Verhalten
BorderLayout	Hauptfenster (JFrame). Platziert Sub-Panels am Rand und im Zentrum.	Zentrum (CENTER) dehnt sich in alle Richtungen maximal aus. Ränder kollabieren bei Nichtnutzung.
FlowLayout	Button-Leisten (unten) oder einfache einzeilige Elemente.	Komponenten behalten Wunschgröße, fließen von links nach rechts und brechen automatisch um.
GridLayout	Eingabemasken (Formulare) mit Labels und Textfeldern.	Strikte Tabelle. Jede Zelle ist exakt gleich groß! Dehnt Komponenten rücksichtslos aus.

2. Verschachtelte Layouts (Das Standard-Klausurszenario)

Häufige Aufgabenstellung: Links ein Grafikfeld, in der Mitte ein strukturiertes Formular und unten eine Reihe Kontrollknöpfe. Das wird durch Verschachtelung gelöst.

Visuelle Struktur der Verschachtelung (Mockup)



Zugehöriger Code für die Verschachtelung (Klasse Gui)

```
import javax.swing.*;
import java.awt.*;

public class Gui extends JFrame {
    // 1. Instanziierung der einzelnen Sub-Panels
    private GrafikPanel gp = new GrafikPanel();           // Eigenes Zeichenpanel
    private EingabePanel ep = new EingabePanel();          // Das Formular-Panel (siehe unten)
    private ButtonPanel bp = new ButtonPanel(ep);          // Das Button-Panel unten

    public Gui() {
        super("Klausur-Anwendung v2");
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        this.setSize(450, 200);
        this.setLocationRelativeTo(null);

        // 2. Das Hauptfenster bekommt das BorderLayout
        this.setLayout(new BorderLayout(10, 10));

        // 3. Einbetten der Sub-Panels in die Himmelsrichtungen
        this.add(this.gp, BorderLayout.WEST);              // Grafik links
        this.add(this.ep, BorderLayout.CENTER);             // Formular füllt die Mitte komplett aus
        this.add(this.bp, BorderLayout.SOUTH);             // Buttons unten verankert

        this.setVisible(true);
    }
}
```

3. Code der verschachtelten Sub-Panels

Hier siehst du, wie die Sub-Panels intern aufgebaut sind, damit sie sich nahtlos in das Hauptfenster einfügen.

Das Formular-Panel in der Mitte (Nutzt intern GridLayout)

```
import javax.swing.*;
import java.awt.*;

public class EingabePanel extends JPanel {
    private JTextField tfName = new JTextField(15);
    private JTextField tfDatum = new JTextField(10);

    public EingabePanel() {
        // Internes Layout festlegen: 2 Zeilen, 2 Spalten, 5px Abstände
        this.setLayout(new GridLayout(2, 2, 5, 5));

        // Zeile 1
        this.add(new JLabel("Name:"));
        this.add(this.tfName);

        // Zeile 2
        this.add(new JLabel("Datum:"));
        this.add(this.tfDatum);
    }

    // Getter-Methoden, damit Knöpfe an die Daten kommen
    public String getNameInput() { return this.tfName.getText().trim(); }
    public String getDatumInput() { return this.tfDatum.getText().trim(); }
}
```

Das Button-Panel unten (Nutzt intern FlowLayout)

```
import javax.swing.*;
import java.awt.*;

public class ButtonPanel extends JPanel {
    private JButton btnAdd = new JButton("Hinzufügen");
    private JButton btnDel = new JButton("Löschen");
    private EingabePanel eingabePanel; // Referenz für Interaktionen

    public ButtonPanel(EingabePanel ep) {
        this.eingabePanel = ep;

        // Knöpfe rechtsbündig anordnen mit Abständen
        this.setLayout(new FlowLayout(FlowLayout.RIGHT, 10, 5));

        // Komponenten einfügen
        this.add(this.btnAdd);
        this.add(this.btnDel);

        // Listener-Anbindung (Muster siehe vorherigen Spickzettel)
        // this.btnAdd.addActionListener(new ButtonListener());
    }
}
```

GOLDENE REGEL BEIM CODEN IM BORDERLAYOUT:

Wenn du eine Komponente (oder ein Sub-Panel) zu einem Container mit BorderLayout hinzufügst und die Himmelsrichtung vergisst (z. B. fälschlicherweise `this.add(panel);` statt `this.add(panel, BorderLayout.SOUTH);` schreibst), packt Java es automatisch in das CENTER. Dadurch wird eine eventuell dort bereits vorhandene Komponente komplett unsichtbar überschrieben! Schreibe die Himmelsrichtung **immer** explizit dazu.

Tipp für Größenanpassungen im BorderLayout:

- Komponenten im NORTH und SOUTH werden auf die volle **Breite** gestreckt (Höhe bleibt wie gewünscht).
- Komponenten im WEST und EAST werden auf die volle **Höhe** gestreckt (Breite bleibt wie gewünscht).
- Das CENTER wird in **beide Richtungen radikal gestreckt**, um jeden freien Pixel auszufüllen.